AtliQ Commerce End-to-End Batch Data Engineering Project

Detailed Implementation, Architecture, Issues & Learning Report

Item	Details
Project type	End-to-end batch data engineering capstone
Cloud stack	Azure + Azure Data Factory + Azure Databricks + Microsoft Fabric
Transformation	PySpark / Databricks + dbt Core
BI	Microsoft Fabric / Power BI

1. Executive Summary

This project implements a complete batch data engineering solution for AtliQ Commerce. Operational commerce data is sourced from Azure SQL, ingested through Azure Data Factory into ADLS Gen2, processed through Bronze and Silver in Azure Databricks, transformed into business-ready Gold models using dbt, automated as a nightly flow, and exposed to Microsoft Fabric for analytics.

The architecture intentionally separates OLTP from analytical processing. Azure SQL remains the operational source of truth, while the downstream lakehouse/analytical layers provide durable, transformed and reporting-oriented data.

The main engineering milestones have been implemented and validated. The current stage is evidence packaging and final repository documentation.

2. Project Milestones

Milestone	Work completed	Status
M1 – OLTP	Azure SQL schema, ETL control/watermark table, daily transaction simulator	Done
M2 – Ingestion	ADF SQL/CSV ingestion and orchestration	Done
M3 – Silver	Databricks Bronze/Silver processing and incremental facts	Done
M4 – Gold/dbt	dbt staging/intermediate/marts and tests	Done
M5 – Nightly sync	Master orchestration, 2 AM trigger, retry/idempotency design	Implemented; proof being packaged
M6 – Fabric	Gold availability and dashboard	Done per project progress
M7 – CI/CD	GitHub repository, GitHub Actions, dbt CI	Core done; proof being packaged

3. End-to-End Architecture

Azure SQL OLTP + CSV files ↓ Azure Data Factory ↓ ADLS Gen2 / Bronze landing ↓ Azure Databricks – Bronze / Silver ↓ dbt – Gold analytical models ↓ Microsoft Fabric / Power BI

ADF orchestration uses pl_master_batch, with pl_sql_to_raw → pl_sql_to_adls and the downstream Databricks_Bronze_to_Gold job.

Why the layers exist

OLTP: optimized for operational transactions and source-system integrity.

Bronze: durable landing boundary that preserves source-oriented data before transformation.

Silver: cleansing, typing, standardization, joins and enrichment.

Gold: business-ready dimensions, facts and marts.

Fabric/Power BI: consumes curated analytical data rather than repeatedly querying the operational source.

4. Source Data and Integration

4.1 Azure SQL

customers

products

orders

order_items

payments

4.2 CSV business datasets

marketing_spend.csv: spend_date, channel, campaign, spend_amount, clicks.

supplier_price_list.csv: product_id, product_name, supplier_name, supplier_cost, effective_date.

The SQL data and the two business datasets were incorporated into the downstream analytical solution. Supplier pricing enriches product/sales information through product_id. Marketing spend is retained as a separate fact/mart because there is no sale-to-campaign attribution key; a direct date-only sales/marketing join could multiply rows and inflate metrics.

5. M1 – OLTP Layer

The operational database is Azure SQL database atliq_commerce on Azure SQL Server atliq-sql-jd.

Created customers, products, orders, order_items and payments.

Created an ETL control/watermark mechanism for incremental extraction.

Implemented a daily transaction simulator to create changing source data for pipeline testing.

This establishes a realistic operational source and allows the downstream pipeline to demonstrate incremental processing rather than only a one-time historical load.

6. M2 – Azure Data Factory

ADF is used for ingestion and orchestration.

Pipelines

pl_sql_to_raw – SQL extraction/landing.

pl_sql_to_adls – lands/moves data into ADLS-oriented Bronze storage.

pl_master_batch – master orchestration.

Read source data using the ingestion/control logic.

Land the data into ADLS Gen2.

Trigger downstream Databricks processing.

Monitor pipeline runs for success/failure and duration.

ADF pipeline JSON representations are being collected as M2 repository evidence.

7. M3 – Databricks Bronze and Silver

Azure Databricks provides distributed transformation. Unity Catalog organizes the project assets under catalog atliq, including bronze, silver and gold schemas.

Dimensions are rebuilt/overwritten where appropriate for a clean current state.

Sales facts use incremental MERGE logic based on the business key.

run_date supports batch traceability.

Product and supplier information is incorporated during enrichment.

Fact grain

fact_sales is one row per order item. Key fields include order_item_id, order_id, customer_id, product_id, order_date, status, quantity, item_price, gross_revenue, product/category information and supplier cost.

8. M4 – dbt Gold

dbt Core manages SQL transformations, model dependencies, testing and documentation while Databricks provides the analytical execution/storage environment.

Model structure

Staging: stg_customers, stg_marketing_spend, stg_order_items, stg_orders, stg_payments, stg_products, stg_supplier_price_list.

Intermediate: int_sales_enriched.

Mart: dim_customer, dim_date, dim_product, fact_sales, fact_marketing_spend.

The current physical implementation materializes staging and intermediate models as views in atliq.gold and mart models as tables. The file dim_products.sql produces the dim_product model.

Validation results

dbt ls: 13 models, 18 data tests, 7 sources.

dbt run: successful twice; 13/13 models passed.

dbt test: 18/18 tests passed; 0 warnings, 0 errors, 0 skips.

Tests include not-null, uniqueness and relationship checks.

Validation	Result
Customer duplicate check	0 duplicates
Product duplicate check	0 duplicates
Fact order-item uniqueness	0 duplicates
Date referential integrity	0 orphan rows
Supplier cost completeness	798 / 798
Revenue	2,126,260.00
Supplier cost	656,785.28

Validation	Result
Gross profit	1,469,474.72
Profit reconciliation difference	0.00

9. M5 – Nightly Automation

The nightly process is designed to ingest changed data, process Silver incrementally, build Gold and complete automatically.

Master pipeline: pl_master_batch.

Databricks job: Databricks_Bronze_to_Gold.

Nightly trigger is configured on pl_sql_to_raw at 2:00 AM.

The end-to-end master execution was successfully verified, including SQL-to-raw, SQL-to-ADLS and Databricks transformation.

Idempotency design

The guide requires running the full nightly job twice and proving that fact_sales row count and total gross_revenue are unchanged after the retry. The implementation is designed for this through same-day Bronze overwrite behavior, Silver MERGE by business key and rebuilding Gold from Silver.

Final evidence still needs to show the before/after figures explicitly.

10. M6 – Microsoft Fabric

The curated Gold data was made available for Fabric reporting and the dashboard work was completed according to the project progress.

Revenue trend by month.

Top products/categories.

Top cities by customer city.

New vs returning customers by signup cohort and order month.

Date/category filtering where applicable.

Cancelled orders are excluded from revenue and Returned activity is treated separately.

11. M7 – Git, CI/CD and Reliability

The project is version-controlled in GitHub. GitHub Actions validates dbt changes.

Repository contains the implementation and dbt project.

GitHub Actions runs dbt CI for pull-request validation.

Databricks host, HTTP path and token are supplied through GitHub Secrets.

The latest CI run succeeded after the Databricks token secret was updated.

At the time evidence packaging began, git status showed a clean working tree because the newly created evidence directories were empty; Git does not track empty directories.

12. Issues Faced and Resolutions

Issue	What happened	Resolution / learning
PowerShell execution policy	Script execution was blocked during local dbt setup.	Used process-scoped Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned.
dbt–Databricks authentication	Initial dbt connection did not work.	Configured environment variables in profiles.yml; corrected the host to omit https:// and replaced the token.
Credential exposure risk	A Databricks token was accidentally pasted during troubleshooting.	Treated it as compromised and replaced it. Current design uses environment variables/GitHub Secrets.
GitHub Actions failure	CI initially failed because the Databricks authentication secret was stale/invalid.	Updated DATABRICKS_TOKEN in GitHub Secrets and reran CI successfully.
Catalog naming	Correct catalog name is atliq; a spelling mismatch can cause object-not-found errors.	Standardized the project on atliq.
Sales + marketing integration	A date-only join could multiply sales rows because campaigns/channels are not uniquely attributable to orders.	Kept marketing spend as a separate fact/mart instead of creating misleading attribution.
Evidence folders	Git reported clean status while evidence folders were empty.	Screenshots/JSON are now being collected before git add/commit.

13. Key Technical Learnings

ADF handles ingestion/orchestration; Databricks handles distributed transformation.

ADLS Gen2 provides durable cloud storage independent of compute availability.

Unity Catalog organizes and governs data assets.

Bronze is source-oriented, Silver is standardized/enriched, and Gold is business-facing.

dbt manages SQL transformation dependencies, tests and documentation; it does not replace the Databricks storage/compute platform.

Fact grain must be explicit: fact_sales is one row per order item.

MERGE is important for incremental processing and retry safety.

Business metric definitions must be explicit; Cancelled orders are excluded from revenue in this project.

Marketing attribution requires a valid attribution key.

Secrets should never be hard-coded or committed.

14. Repository Evidence Structure

Soln/ ■■■ dbt_project/ ■■■ .github/workflows/ci.yml ■■■ evidence/ ■ ■■■ M1_OLTP/ ■ ■■■ M2_ADF/ ■ ■■■ M3_SILVER/ ■ ■■■ M4_GOLD_DBT/ ■ ■■■ M5_NIGHTLY_SYNC/ ■ ■■■ M6_FABRIC/ ■ ■■■ M7_CICD/ ■■■ docs/

15. Remaining Work

■ Finish the remaining M4 dbt evidence screenshots.

■ Capture M5 nightly trigger proof showing 2:00 AM.

■ Capture M5 idempotency proof: before/after fact_sales row count and gross_revenue after a complete retry.

■ Ensure M6 dashboard screenshots are stored in evidence/M6_FABRIC.

■ Store M7 GitHub Actions success proof.

■ Review evidence files for secrets before committing.

■ Run git status, inspect the exact files, then git add/commit/push.

■ Perform a final submission audit against the project guide.

16. Interview / Client Explanation

I built an end-to-end nightly batch data engineering pipeline for an e-commerce workload. Azure SQL acts as the OLTP source. Azure Data Factory performs ingestion and orchestration, landing data into ADLS Gen2. Azure Databricks processes the data through Bronze and Silver, using incremental MERGE logic for the sales fact. dbt Core builds and tests the Gold analytical models in Databricks. The curated data is exposed to Microsoft Fabric for BI. The solution is version-controlled in Git, with GitHub Actions validating dbt changes. I also validated data quality, financial reconciliation and retry/idempotency behavior so the pipeline is repeatable and production-oriented.

17. Final Note

This report is a learning and implementation record based on the supplied project guide and the work completed in the project. Where a guide requirement still needs explicit screenshot evidence, it is marked as remaining rather than being presented as proven.